

EduFocus

Méthodes statistiques, apprentissage machine et data mining

Un cours appliqué : les mathématiques derrière l'analyse

de l'exclusion scolaire en Mauritanie

Projet EduFocus – Datathon IndabaX Mauritanie 2026

*Ce document est un cours qui explique **pourquoi** et **comment** chaque méthode est utilisée dans EduFocus : d'abord l'idée, ensuite la **formule mathématique**, puis **où** elle intervient dans le code, l'API et l'interface.*

Table des matières

1	Introduction : le problème et les trois familles de méthodes	2
2	Statistiques descriptives et inférentielles	2
2.1	Normalisation min-max	2
2.2	Standardisation z-score	2
2.3	Transformation logarithmique	3
2.4	Médiane et partage en quadrants	3
2.5	Corrélation de Pearson	3
2.6	Concentration : Pareto, Lorenz et indice de Gini	4
2.7	Tendance et régression linéaire simple (moindres carrés)	4
3	Apprentissage machine non supervisé : typologie des wilayas	5
3.1	K-means	5
3.2	Choisir k : inertie (coude) et silhouette	5
4	Apprentissage machine supervisé : les déterminants individuels	6
4.1	Régression logistique	6
5	Data mining : règles d'association (Apriori)	7
6	Analyse de graphes et détection de communautés	8
6.1	Représentation en graphe	8
6.2	Détection de communautés : Louvain	8
6.3	Layout « force-directed » (carte du graphe)	9
7	Indicateur composite : l'Indice de Priorité Éducative (IPE)	9
8	Scénarios prospectifs à 2030	9
9	Récapitulatif : méthode $\leftrightarrow$ où c'est utilisé	10
10	Bibliothèque logicielle	11

1 Introduction : le problème et les trois familles de méthodes

EduFocus analyse l'exclusion scolaire en Mauritanie : **375 566 enfants de 6–14 ans** (soit **33,1 %**) sont hors de l'école formelle selon l'enquête EPCV 2019, sur une population cible de 1 135 657 enfants répartis dans les **13 wilayas**.

Pour transformer ces données brutes en décisions d'investissement, le projet utilise trois familles de méthodes complémentaires :

1. **Statistiques** : décrire (moyennes, médianes, concentration) et mesurer les liaisons entre indicateurs (corrélation de Pearson).
2. **Apprentissage machine** : *non supervisé* pour regrouper les wilayas (K-means, silhouette) et *supervisé* pour expliquer les déterminants individuels (régression logistique).
3. **Data mining et graphes** : règles d'association (Apriori) et analyse de réseaux (détection de communautés de Louvain) pour découvrir des structures non évidentes.

Fil rouge. Chaque méthode est décrite dans ce cours comme suit : *idée* → *formule* → *exemple chiffré* → où c'est utilisé. Un tableau récapitulatif complet est donné au § 9.

2 Statistiques descriptives et inférentielles

2.1 Normalisation min–max

Idée. Les indicateurs n'ont pas la même unité (un taux en %, un effectif en milliers, un ratio sans dimension). Pour les comparer et les combiner, on les ramène sur une échelle commune [0, 100].

Formule

$$x'_i = \frac{x_i - \min(x)}{\max(x) - \min(x)} \times 100$$

où $\min(x)$ et $\max(x)$ sont les valeurs extrêmes observées sur les 13 wilayas. Si $\max(x) = \min(x)$ (pas de variation), on fixe $x'_i = 50$.

Où c'est utilisé dans EduFocus

backend/analytics/index.py – fonction `normalize()` : normalise le **volume** (nombre d'enfants hors école, en échelle log, cf. 2.3) et l'**intensité** (taux de hors-école) avant de construire l'indice IPE (§ 7). Accessible via `/api/wilayas` et affiché sur la page `/wilayas`, la carte `/carte` et le rapport `/rapport`.

2.2 Standardisation z-score

Idée. Pour un *clustering*, les données doivent être centrées et réduites : chaque variable doit avoir la même échelle, sinon les variables grandes en valeur domineraient les calculs de distance.

Formule

$$z_i = \frac{x_i - \mu}{\sigma}, \quad \mu = \frac{1}{n} \sum_{i=1}^n x_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}.$$

Après transformation : moyenne 0, écart-type 1.

Où c'est utilisé dans EduFocus

`backend/analytics/clustering.py` – `StandardScaler().fit_transform(...)` sur 8 features (intensité, pauvreté, ruralité, dépendance jeunes, mahadra, aucune instruction, écart de genre, densité d'écoles) avant le K-means (§ 3.1).

2.3 Transformation logarithmique

Idée. Le nombre d'enfants hors école varie de quelques milliers (Nouadhibou) à des centaines de milliers (Hodh Ech Chargui). Une échelle logarithmique « compresse » les grands nombres et rend la variable symétrique.

Formule

$$y = \log(1 + x) \quad (\log1p)$$

Le +1 garantit que $x = 0 \mapsto y = 0$. Utilisée avant la normalisation min-max du *volume* et dans la matrice de stratégie.

Où c'est utilisé dans EduFocus

`backend/analytics/index.py` (`np.log1p`) et `backend/analytics/matrice.py` (axe « volume » du scatter). Page `/strategies`.

2.4 Médiane et partage en quadrants

Idée. La médiane coupe une distribution en deux moitiés égales. Elle est robuste aux valeurs extrêmes (contrairement à la moyenne).

Définition

Pour un échantillon trié $x_{(1)} \leq \dots \leq x_{(n)}$:

$$\text{médiane} = \begin{cases} x_{(\frac{n+1}{2})} & \text{si } n \text{ impair} \\ \frac{1}{2}(x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}) & \text{si } n \text{ pair.} \end{cases}$$

Où c'est utilisé dans EduFocus

`backend/analytics/matrice.py` : chaque wilaya est classée *haut/bas* selon la médiane du volume log et la médiane de l'intensité → **4 quadrants** (« Effort massif », « Ciblage des effectifs », « Traitement local », « Consolidation »). Page `/strategies`, endpoint `/api/matrice`.

2.5 Corrélation de Pearson

Idée. Mesure la force et le sens de la *liaison linéaire* entre deux indicateurs, en unités normalisées (donc comparable entre paires).

Formule

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} = \frac{\text{Cov}(x, y)}{\sigma_x \sigma_y}$$

Propriétés : $r \in [-1, 1]$; $r = 1$ (resp. -1) : corrélation linéaire positive (resp. négative) parfaite; $r = 0$: aucune liaison linéaire.

Ce que r mesure et ne mesure pas. La corrélation est *symétrique* et *sans direction de causalité* : si le taux de pauvreté et le taux de hors-école corrélient à $r = 0,84$, cela ne prouve pas que la pauvreté *cause* le hors-école. C'est le rôle de la régression logistique (§ 4.1) d'approcher la causalité en *ajustant* les facteurs.

Où c'est utilisé dans EduFocus

Trois usages :

- `backend/analytics/indicateurs.py` : matrice complète 12×12 affichée en heatmap sur `/indicateurs` et `/rapport` (endpoint `/api/indicateurs`). Paires les plus fortes : dépendance jeunes $\times$ part 0–14 ans ($r = 1,00$), aucune instruction $\times$ chef de ménage sans éducation ($r = 0,96$), ruralité $\times$ dépendance jeunes ($r = 0,91$).
- `backend/analytics/graph.py` : réseau de similarité entre wilayas ($|r| > 0,85$, top-2 voisins) et graphe de corrélations entre indicateurs ($|r| > 0,75$). Page `/reseau`.
- `backend/analytics/concentration.py` : n'utilise pas r mais quantifie la concentration (voir § 2.6).

2.6 Concentration : Pareto, Lorenz et indice de Gini

Idée. Le hors-école n'est pas uniformément réparti. On veut savoir **combien de wilayas portent la moitié du problème** et mesurer l'inégalité de la répartition (loi de Pareto « 20 % des zones $\rightarrow$ 80 % du problème »).

Formules

Courbe de Lorenz (part cumulée) : on trie $x_{(1)} \geq \dots \geq x_{(n)}$ par ordre décroissant, alors

$$\text{Lorenz}(k) = \frac{\sum_{i=1}^k x_{(i)}}{\sum_{i=1}^n x_{(i)}} \times 100.$$

Indice de Gini (formule par paires, utilisée dans le code) :

$$G = \frac{\sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|}{2n \sum_{i=1}^n x_i}.$$

Interprétation : $G = 0$ répartition parfaitement égale, $G = 1$ concentration totale. G est lié à l'aire entre la courbe de Lorenz et la diagonale (égalité parfaite).

Où c'est utilisé dans EduFocus

`backend/analytics/concentration.py` : calcule la part cumulée des 5 premières wilayas (**top5**), le nombre de wilayas pour atteindre 50 % du problème, la courbe de Lorenz et le Gini. Endpoints `/api/concentration`; affiché sur `/` (Home), `/strategies` et `/rapport`.

2.7 Tendances et régression linéaire simple (moindres carrés)

Idée. Pour projeter le taux de hors-école jusqu'en 2030, on mesure la pente de la tendance passée (World Bank, 2008–2024) par régression linéaire.

Formule

Modèle $y = \beta_0 + \beta_1 x + \varepsilon$. Le moindre carré minimise $\sum_i (y_i - \beta_0 - \beta_1 x_i)^2$, solution :

$$\beta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad \beta_0 = \bar{y} - \beta_1 \bar{x}.$$

Où c'est utilisé dans EduFocus

`backend/analytics/scenarios.py` – `np.polyfit(x, y, 1)` : pente annuelle bornée dans $[-1, 0, 0]$ points par an, prolongée sur 2030 – 2022 ans. Endpoint `/api/scenarios`, page `/strategies`. Les séries brutes WDI sont servies par `/api/trends` sur `/tendances`.

3 Apprentissage machine non supervisé : typologie des wilayas

3.1 K-means

Idée. Regrouper les 13 wilayas en k classes homogènes selon leurs 8 indicateurs normalisés. Deux wilayas d'une même classe ont des profils d'exclusion proches $\Rightarrow$ la même stratégie d'investissement.

Algorithme et objectif

On minimise l'*inertie* (somme des distances quadratiques intra-classe) :

$$J = \sum_{c=1}^k \sum_{i \in C_c} \|x_i - \mu_c\|^2.$$

Algorithme itératif (Lloyd) :

1. initialiser k centres μ_c aléatoirement ;
2. **affectation** : $C_c \leftarrow \{i : c = \arg \min_c \|x_i - \mu_c\|\}$;
3. **mise à jour** : $\mu_c \leftarrow \frac{1}{|C_c|} \sum_{i \in C_c} x_i$;
4. répéter 2–3 jusqu'à convergence (inertie stable).

Où c'est utilisé dans EduFocus

`backend/analytics/clustering.py` : `KMeans(n_clusters=k, n_init=20)` sur données z-scorées. $k = 3$ retenu pour une typologie lisible (modérée / dominante « mahadra » / dominante « aucune instruction »). Endpoint `/api/clusters`, pages `/clusters`, `/wilayas` et `/rapport`.

3.2 Choisir k : inertie (coude) et silhouette

Idée. L'inertie $J(k)$ décroît toujours avec k ; le bon k est celui où la courbe « casse » (méthode du coude). La *silhouette* mesure la cohésion d'un point avec sa classe par rapport à la classe voisine.

Silhouette

Pour chaque point i :

$a(i)$ = distance moyenne aux points de sa propre classe,

$b(i)$ = min des distances moyennes aux autres classes,

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1].$$

$s(i) \approx 1$: point bien classé ; $s(i) < 0$: point probablement mal affecté. On choisit le k qui maximise la *silhouette moyenne*.

Où c'est utilisé dans EduFocus

`backend/analytics/clustering.py` – `choose_k()` balaye $k \in [2, 6]$, calcule inerties + `silhouette_score` ; le k optimal est ensuite raffiné pour la lisibilité (voir la note dans `run_all.py`).

4 Apprentissage machine supervisé : les déterminants individuels

4.1 Régression logistique

Idée. Prédire une variable *binnaire* – `hors_ecole` (0/1) pour chaque enfant de 6–14 ans de l'EPCV (16 451 enfants) – en fonction de facteurs : sexe, âge, milieu, pauvreté, éducation du chef de ménage, wilaya.

Modèle

La probabilité d'être hors école s'écrit avec la fonction logistique :

$$p = P(y = 1 | x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}}.$$

Ou, en logit (log des cotes) :

$$\log \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p.$$

Les β sont estimés par **maximum de vraisemblance** : on maximise $L(\beta) = \prod_i p_i^{y_i} (1 - p_i)^{1-y_i}$.

Odds-ratio et intervalle de confiance

Le *odds-ratio* de la variable x_j est

$$OR_j = e^{\beta_j},$$

avec intervalle de confiance à 95 % $[e^{\beta_j - 1,96 SE_j}, e^{\beta_j + 1,96 SE_j}]$.

- $OR > 1$: le facteur *augmente* la probabilité d'exclusion ;
- $OR < 1$: il la *réduit* ;
- tous les effets sont *ajustés* : « toutes choses égales par ailleurs ».

Où c'est utilisé dans EduFocus

`backend/analytics/logit.py` : `sm.Logit(...)` de `statsmodels`, avec variables binaires (références : garçon, 10–14 ans, urbain, non pauvre, Nouakchott). Résultats : `odds_ratio`, `IC`, `pvalue`, `pseudo_r2`, `aic`. Endpoint `/api/logit`, affiché sur `/` (Home) et `/rapport`. Exemple de lecture : un enfant en milieu rural a un $OR > 1$ par rapport à l'urbain, à *sexe*, *âge*, *pauvreté* et *wilaya égaux*.

Séparation complète et pseudo- R^2

Le facteur « chef de ménage sans éducation » est *parfaitement prédictif* (tout enfant concerné est hors école) : la vraisemblance n'a pas de maximum fini, le coefficient n'est pas estimable. Le code le détecte et le traite par les règles d'association (confiance 100 %, lift 3,02, cf. § 5).

$$\text{pseudo-}R_{\text{McFadden}}^2 = 1 - \frac{\ln L(\text{modèle complet})}{\ln L(\text{modèle constant})}, \quad \text{AIC} = 2p - 2 \ln L.$$

5 Data mining : règles d'association (Apriori)

Idée. Trouver des *combinaisons de facteurs* fréquemment associées au hors-école, sans hypothèse a priori : milieu rural, pauvreté, absence d'éducation du chef de ménage, région → hors école.

Support, confiance, lift

Pour une règle $A \Rightarrow B$ (ici $B = \{\text{hors école}\}$) :

$$\text{support}(A \Rightarrow B) = \frac{\text{freq}(A \cup B)}{N}, \quad \text{confiance}(A \Rightarrow B) = \frac{\text{support}(A \cup B)}{\text{support}(A)},$$

$$\text{lift}(A \Rightarrow B) = \frac{\text{confiance}(A \Rightarrow B)}{\text{support}(B)}.$$

- *Support* : fréquence de la combinaison dans la population ;
- *Confiance* : probabilité de B sachant A ;
- *Lift* : rapport à l'indépendance. $\text{lift} > 1$: A rend B *plus probable* qu'au hasard ; c'est le critère de pertinence utilisé ici (seuil $> 1,05$).

Algorithme Apriori

1. générer les *itemsets* de taille 1 dont le support dépasse `min_support` ;
2. construire itérativement les itemsets de taille $k + 1$ par *jointure* d'itemsets de taille k (propriété d'anti-monotonie : tout sur-ensemble d'un itemset non fréquent est non fréquent) ;
3. produire les règles et ne garder que celles satisfaisant `min_confidence` et un $\text{lift} > 1$.

Où c'est utilisé dans EduFocus

`backend/analytics/rules.py` : `apriori()` et `association_rules()` de `mlxtend` sur les 16 451 enfants. Filtres : conséquence = exactement `\{hors_ecole\}`, $\text{lift} > 1,05$. Endpoints : `/api/rules`. **Page utilisée** : `/regles` (page dédiée du frontend), qui affiche les règles triées

par lift, les règles sans la variable « chef de ménage » (*sans_cm*) et les règles par région (*par_region*). Résultat : la variable « chef de ménage sans éducation » produit une règle à confiance 100 % et lift 3,02 – c’est le facteur individuel le plus fort, là où la régression logistique ne pouvait pas l’estimer.

6 Analyse de graphes et détection de communautés

6.1 Représentation en graphe

Idée. Un *graphe* $G = (V, E)$ est un ensemble de nœuds V (wilayas ou indicateurs) reliés par des arêtes E pondérées (par $|r|$). La structure du graphe révèle des groupes, des hubs et des isolés invisibles dans un tableau.

Degré et matrice d’adjacence

Le *degré* $\deg(v)$ d’un nœud est le nombre de ses voisins. La *matrice d’adjacence* A vaut $A_{uv} = 1$ si $(u, v) \in E$, et la matrice *pondérée* $W_{uv} = |r_{uv}|$ sinon 0. Le degré (pondéré) est $\deg(v) = \sum_u W_{uv}$. Dans EduFocus, le degré des indicateurs détermine la *taille des nœuds* du graphe.

Où c’est utilisé dans EduFocus

backend/analytics/graph.py :

- **Réseau de similarité des wilayas** : nœuds = wilayas, arêtes = paires dont $|r| \geq 0,85$ (top-2 voisins), poids = $|r|$.
- **Graphe de corrélations des indicateurs** : nœuds = indicateurs, arêtes si $|r| \geq 0,75$, taille des nœuds $\propto$ degré. Endpoints `/api/graph/similarite` et `/api/graph/correlations` ; page `/reseau`.

6.2 Détection de communautés : Louvain

Idée. Regrouper les wilayas qui partagent le même profil d’exclusion. La méthode de *Louvain* maximise la *modularité* Q .

Modularité de Newman–Girvan

$$Q = \frac{1}{2m} \sum_{u,v} \left[W_{uv} - \frac{d_u d_v}{2m} \right] \delta(c_u, c_v),$$

avec $m = \frac{1}{2} \sum_{u,v} W_{uv}$ (poids total des arêtes), d_u le degré pondéré de u , et $\delta(c_u, c_v) = 1$ si u et v sont dans la même communauté. $Q \in [-1, 1]$; $Q > 0,3$ indique une structure communautaire réelle (meilleure que le hasard).

Algorithme de Louvain

1. **Phase locale** : chaque nœud est déplacé vers la communauté voisine qui augmente le plus Q (gain de modularité), jusqu’à stabilisation ;
2. **Agrégation** : les communautés deviennent des super-nœuds ;
3. on répète 1–2 jusqu’à convergence.

Où c'est utilisé dans EduFocus

backend/analytics/graph.py : community_louvain.best_partition(G, random_state=42) (python-louvain). Résultat : **3 communautés** de wilayas au profil proche, exposées sur /reseau (couleurs des nœuds) et dans le rapport PDF /rapport.

6.3 Layout « force-directed » (carte du graphe)

Idée. Positionner les nœuds dans le plan en simulant un système de forces physique : les arêtes tirent les nœuds (ressorts), les nœuds se repoussent (répulsion de Coulomb).

Force dirigée (modèle ressort)

Force de ressort entre nœuds reliés : $F_s = k_s (d - l_0)$ (Hooke) ; répulsion entre tous les nœuds : $F_r = \frac{k_r}{d^2}$; amortissement η : on itère $v += (F_s + F_r)/m$, $x += \eta v$. La position finale est une lecture visuelle de la structure : wilayas connectées se rapprochent, communautés forment des groupes visibles.

Où c'est utilisé dans EduFocus

Frontend : frontend/src/lib/charts.ts (forceGraph()) avec ECharts (layout *force*). Taille des nœuds wilayas $\propto$ nombre d'enfants hors école ; page /reseau.

7 Indicateur composite : l'Indice de Priorité Éducative (IPE)

Idée. Combiner trois dimensions en *une* note 0–100 pour classer les wilayas : volume du problème, intensité de l'exclusion, vulnérabilité du territoire.

Formule

$$\text{IPE} = 0,40 \text{ Volume} + 0,35 \text{ Intensité} + 0,25 \text{ Vulnérabilité}$$

avec (toutes normalisées en 0–100, voir § 2.1) :

- Volume = $\log(1 + \text{enfants hors école})$ normalisé ;
- Intensité = taux hors école (%) normalisé ;
- Vulnérabilité = $0,75 \text{ pauvreté} + 0,25 \text{ dépendance jeunes}$.

Le rang est la *position décroissante* de l'IPE sur les 13 wilayas (méthode min pour les ex æquo).

Où c'est utilisé dans EduFocus

backend/analytics/index.py (build_index()) : classe les wilayas et dérive un *levier d'action* dominant par règles de seuil (construire des écoles / passerelles mahadra / rétention). Endpoints /api/summary et /api/wilayas ; utilisé sur /, /wilayas, /carte (choroplèthe) et /rapport.

8 Scénarios prospectifs à 2030

Idée. Traduire les leviers en trajectoires chiffrées : tendance « laisser-faire », passerelles mahadra $\rightarrow$ formel (25 % et 50 %), construction d'écoles de proximité.

Projection

Avec $\tau_0 = 33,1\%$ (2022), pente annuelle β (§ 2.7) et $n_y = 2030 - 2022$:

$$\tau_{2030} = \max(\tau_0 + \beta n_y, 0)$$

puis, si l'on convertit une part s des M_0 enfants mahadra (199 493) vers le formel :

$$\tau_{2030} \leftarrow \tau_{2030} - \frac{s M_0}{P_{6-14}} \times 100$$

où $P_{6-14} = 1\,135\,657$. Besoin en écoles : on cible une densité de 1 établissement pour 1 000 enfants, $\text{besoin}_w = \max(1 - d_w, 0) \times \frac{\text{pop}_{6-14}^{(w)}}{1000}$, et coût estimé à 25 M MRO par établissement.

Où c'est utilisé dans EduFocus

`backend/analytics/scenarios.py` : 4 scénarios comparés (taux et effectifs 2030, réduction vs 2022, coûts). Endpoint `/api/scenarios`, page `/strategies` et rapport `/rapport`.

9 Récapitulatif : méthode ↔ où c'est utilisé

Famille	Méthode	Backend	Endpoint	Pages
Stat.	Normalisation min-max	<code>index.py</code>	<code>/api/wilayas</code>	<code>/</code> , <code>/wilayas</code> , <code>/carte</code> , <code>/rapport</code>
Stat.	z-score	<code>clustering.py</code>	<code>/api/clusters</code>	<code>/clusters</code> , <code>/wilayas</code> , <code>/rapport</code>
Stat.	loglp	<code>index.py</code> , <code>matrice.py</code>	<code>/api/wilayas</code> , <code>/api/matrice</code>	<code>/strategies</code>
Stat.	Médianes (quadrants)	<code>matrice.py</code>	<code>/api/matrice</code>	<code>/strategies</code> , <code>/rapport</code>
Stat.	Corrélation de Pearson	<code>indicateurs.py</code> , <code>graph.py</code>	<code>/api/indicateurs</code> , <code>/api/graph/*</code>	<code>/indicateurs</code> , <code>/reseau</code> , <code>/rapport</code>
Stat.	Pareto / Lorenz / Gini	<code>concentration.py</code>	<code>/api/concentration</code>	<code>/</code> , <code>/strategies</code> , <code>/rapport</code>
Stat.	OLS (tendance)	<code>scenarios.py</code>	<code>/api/scenarios</code>	<code>/strategies</code> , <code>/rapport</code>
ML non sup.	K-means + silhouette	<code>clustering.py</code>	<code>/api/clusters</code>	<code>/clusters</code> , <code>/wilayas</code> , <code>/rapport</code>
ML sup.	Régression logistique	<code>logit.py</code>	<code>/api/logit</code>	<code>/</code> , <code>/rapport</code>
DM	Apriori (règles)	<code>rules.py</code>	<code>/api/rules</code>	<code>/regles</code>
Graphe	Louvain (communautés)	<code>graph.py</code>	<code>/api/graph/similarity</code>	<code>/reseau</code>
Graphe	Force-directed (ECharts)	—	—	<code>/reseau</code>
Index	IPE composite	<code>index.py</code>	<code>/api/summary</code>	<code>/</code> , <code>/wilayas</code> , <code>/carte</code> , <code>/rapport</code>
Prospectif	Projections 2030	<code>scenarios.py</code>	<code>/api/scenarios</code>	<code>/strategies</code> , <code>/rapport</code>

TABLE 1 – Correspondance complète méthode → code → API → interface.

Lecture croisée. Les méthodes ne sont pas isolées : la normalisation alimente le clustering et l'IPE ; la corrélation alimente les graphes ; les règles d'association compensent la limite de la régression (séparation complète). L'IPE hiérarchise, le **clustering** regroupe,

les **graphes** révèlent la structure, la **logistique** explique les causes individuelles, et les **scénarios** chiffrent les actions.

10 Bibliothèque logicielle

Bibliothèque	Rôle	Méthodes mobilisées
numpy / pandas	calcul	vecteurs, agrégats, tableaux
scipy	statistiques	linkage (CAH), distributions
scikit-learn	ML	KMeans, StandardScaler, silhouette_score
statsmodels	stats	Logit (régression logistique, IC, p)
networkx	graphes	Graph, degree
python-louvain	graphes	best_partition (Louvain)
mlxtend	data mining	apriori, association_rules
pyreadstat	données	lecture EPCV 2019 (.sav)
fastapi / uvicorn	API	exposition des résultats

TABLE 2 – Bibliothèques Python utilisées par le backend (`backend/requirements.txt`).